

```

;          Write X86/64 ALP to perform multiplication of two
8-bit hexadecimal numbers.
;          Use successive addition and add and shift
method.
;          Use of 64-bit registers is expected.

; ----- Macros for Syscall-based I/O -----
%macro write 2
    mov rax, 1      ; syscall number for write
    mov rdi, 1      ; file descriptor 1 (stdout)
    mov rsi, %1     ; buffer
    mov rdx, %2     ; length
    syscall
%endmacro

%macro read 2
    mov rax, 0      ; syscall number for read
    mov rdi, 0      ; file descriptor 0 (stdin)
    mov rsi, %1     ; buffer
    mov rdx, %2     ; length
    syscall
%endmacro

; ----- Data Section -----
section .data
    dispMsg db "Write X86/64 ALP to perform
multiplication of two 8-bit hexadecimal numbers. Use
successive addition and add and shift method. (Use of 64-
bit registers is expected).",10,"Name:- Sumaanyu",10,"Roll:-
7256",10,"Date of Performance:- 03/04/2025",10
    dispMsgLen equ $-dispMsg

    msg db "-----MENU-----", 10
    db "    1. Successive Addition ", 10
    db "    2. Add and Shift ", 10
    db "    3. Exit ", 10
    db "Enter your choice : "
    msglen equ $-msg

    msg1 db "Number : "
    len equ $-msg1

    endl db 10

; ----- BSS Section -----
section .bss
    choice resb 2    ; to store menu choice
    num resb 3       ; to store user input string
    num1 resb 1      ; to store 1st number (after
conversion)
    num2 resb 1      ; to store 2nd number (after
conversion)
    temp resd 1      ; buffer for displaying result in hex

; ----- Code Section -----
section .text
    global _start

_start :
    write dispMsg, dispMsgLen ; display welcome
message

```

```

main_menu :
    write msg, msglen      ; display menu
    read choice, 2         ; read choice

    cmp byte[choice], "1"  ; check for successive
addition
    je case1

    cmp byte[choice], "2"  ; check for add and shift
    je case2

    cmp byte[choice], "3"  ; exit
    je case3

    jmp main_menu         ; loop back to menu if
invalid input

; ----- Case 1: Successive Addition -----
case1 :
    write msg1, len        ; prompt first number
    read num, 3
    call ascii_hex         ; convert ASCII to hex
    mov [num1], bl

    write msg1, len        ; prompt second number
    read num, 3
    call ascii_hex
    mov [num2], bl

    call successive_addition ; perform
multiplication
    jmp main_menu

; ----- Case 2: Add and Shift -----
case2 :
    write msg1, len
    read num, 3
    call ascii_hex
    mov [num1], bl

    write msg1, len
    read num, 3
    call ascii_hex
    mov [num2], bl

    call add_shift
    jmp main_menu

; ----- Case 3: Exit -----
case3 :
    mov rax, 60            ; syscall: exit
    mov rdi, 0
    syscall

; ----- Procedure: Successive Addition -----
successive_addition:
    mov cl, [num2]         ; multiplier (loop count)
    mov rbx, 0             ; result = 0
    mov al, [num1]         ; multiplicand

```

```

next_add :
    add bx, ax      ; result += multiplicand
    loop next_add   ; repeat for num2 times

    call display    ; display result
    ret

; ----- Procedure: Add and Shift -----
add_shift:
    mov rbx, 0      ; result = 0
    mov al, [num1]   ; multiplicand
    mov dl, [num2]   ; multiplier
    mov cl, 8        ; 8-bit multiplication (8 shifts)

next_shift :
    bt rdx, 0        ; check LSB of multiplier
    jnc ahead        ; if 0, skip addition
    add bx, ax        ; if 1, add multiplicand to result

ahead :
    rol ax, 1        ; shift multiplicand left
    ror dl, 1        ; shift multiplier right
    loop next_shift   ; repeat 8 times

    call display
    ret

; ----- Procedure: Convert ASCII hex input to binary -----
ascii_hex:
    mov bl, 0
    mov rsi, num
    mov rcx, 2        ; 2 digits expected

next :
    rol bl, 4        ; shift result nibble
    mov al, [rsi]
    cmp al, '9'
    jbe sub30h
    sub al, 7h        ; adjust for A-F

sub30h :
    sub al, 30h
    add bl, al
    inc rsi
    loop next
    ret

; ----- Procedure: Display hexadecimal number -----
display :
    mov rsi, temp
    mov rcx, 4        ; display 4 hex digits

next1 :
    rol bx, 4        ; extract higher nibble
    mov al, bl
    and al, 0Fh
    cmp al, 9
    jbe add30h

    add al, 7h

add30h :
    add al, 30h      ; convert to ASCII
    mov [rsi], al
    inc rsi
    loop next1

    write temp, 4
    write endl, 1
    ret

```